

An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

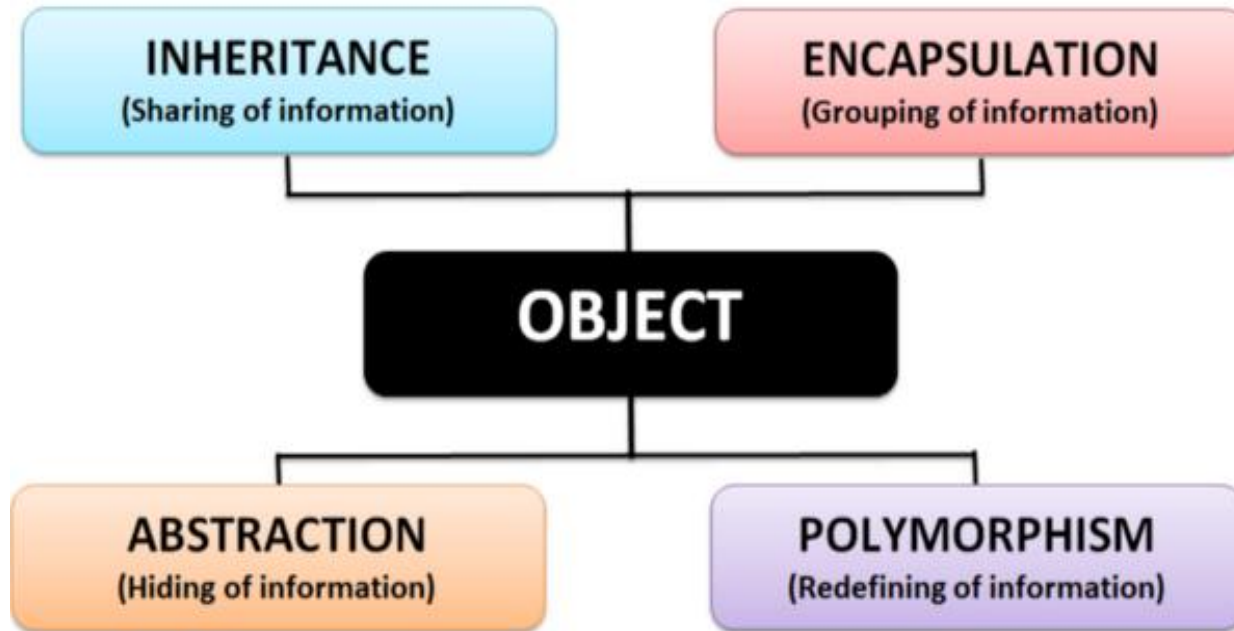
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Single Inheritance & Multiple Inheritance



**Welcome to *the* Object Oriented
Programming *course***

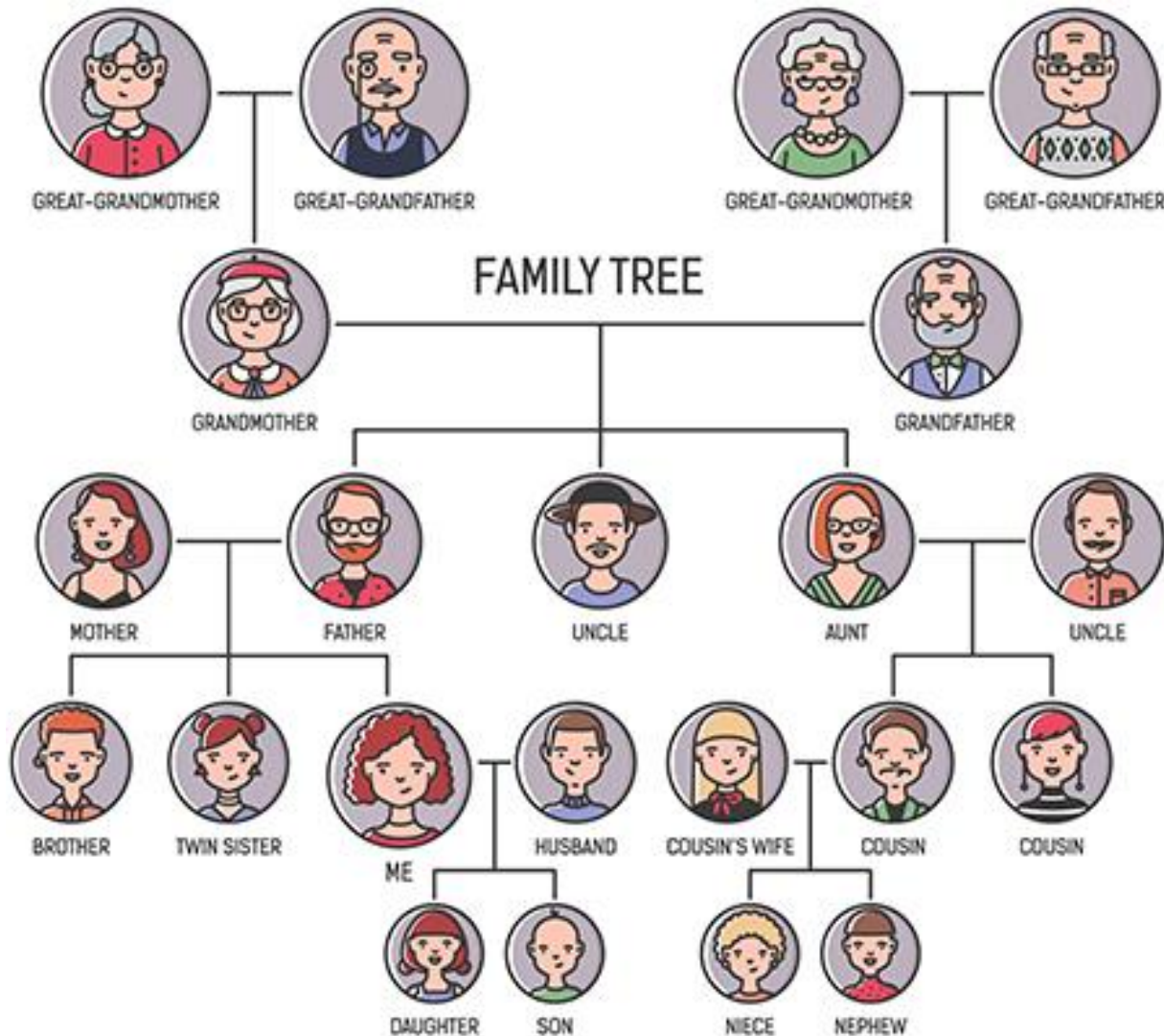
Understanding *the* Concept of OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

Inheritance provides code reusability to the program because we can use an existing class to create a new class instead of creating it from scratch.

Inheritance Concepts

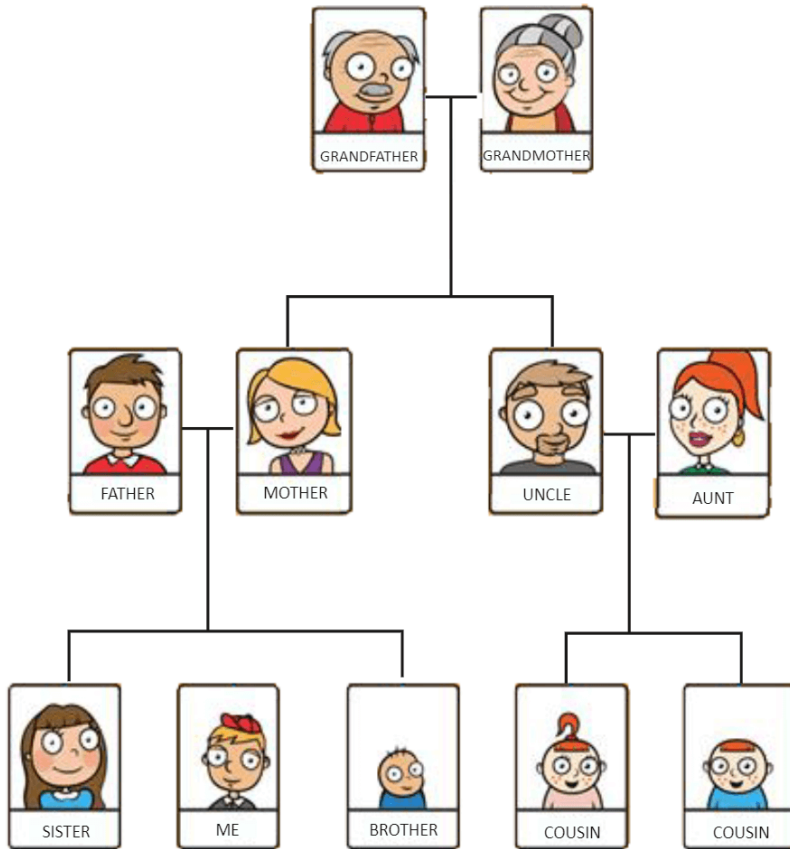


Anak dapat mewarisi kombinasi sifat-sifat dari kedua orangtuanya.

Jika seorang anak mewarisi **rambut keriting** dari salah satu orangtuanya dan **warna rambut** dari orang lainnya, anak tersebut memiliki kombinasi sifat-sifat genetik dari kedua orangtuanya.

What is Inheritance ???

The process of inheriting the properties of the **parent class** into a **child class** is called inheritance.



The existing class is called a base class or parent class and the new class is called a subclass or child class or derived class.

What is Inheritance ???

- ❖ Inheritance is an important aspect of the **OBJECT-ORIENTED PARADIGM**.
- ❖ In inheritance, the child class acquires the properties and can access all the data members and functions defined in the parent class. A child class can also provide its specific implementation to the functions of the parent class.
- ❖ It is a mechanism that **allows us to create a hierarchy of classes that share a set of properties and methods** by deriving a class from another class. Inheritance is the capability of one class to derive or inherit the properties from another class.

Benefits of inheritance are:

- 1) It represents real-world relationships well.
- 2) It provides the reusability of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
- 3) It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.
- 4) Inheritance offers a simple, understandable model structure.
- 5) Less development and maintenance expenses result from an inheritance.

What is Inheritance ???

Inheritance memungkinkan kita untuk mewarisi atribut dan metode dari suatu kelas dasar (*base*) atau induk (*parent*).

Hal ini dapat menjadi sangat berguna karena kita membuat sub-kelas yang dapat menggunakan semua fungsi dari *parent* class.

Selain itu, kita dapat **menimpa** atau **menambahkan** fungsi yang baru tanpa mempengaruhi *parent* class.

Python Has Two Classes

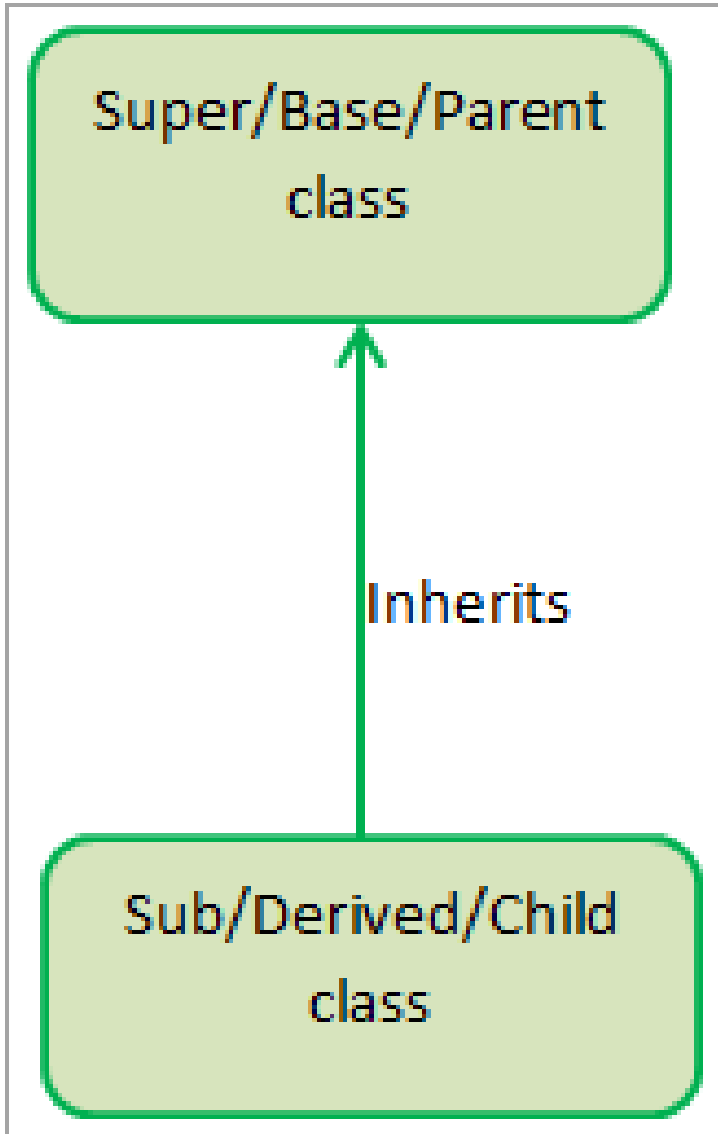
Python yang memiliki dua kelas, yaitu:

- 1) **Parent class** (*super atau base class*)
- 2) **Child class** (*subclass atau derived class*)

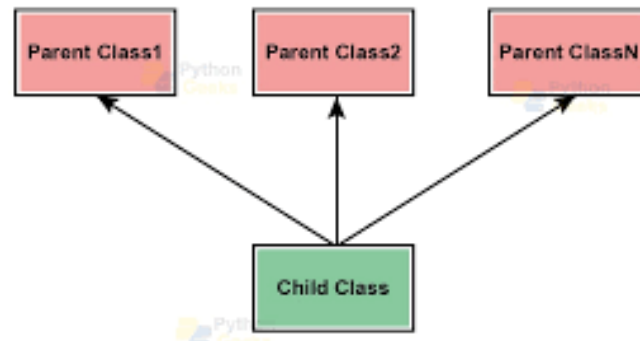
Class yang mewarisi sifat disebut sebagai Child class, sedangkan class yang diwarisi sifatnya adalah Parent class.

Inheritance memiliki kemampuan untuk membuat Sub-classes yang berisi **spesialisasi** dari Parent class.

Parent Class & Child Class *in* Python



Multiple Inheritance in Python



Python Inheritance:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

- 1) **Parent class** is the class being inherited from, also called **base class**.
- 2) **Child class** is the class that inherits from another class, also called **derived class**.

The slide features a white background with green leaves framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

1 - Single Inheritance

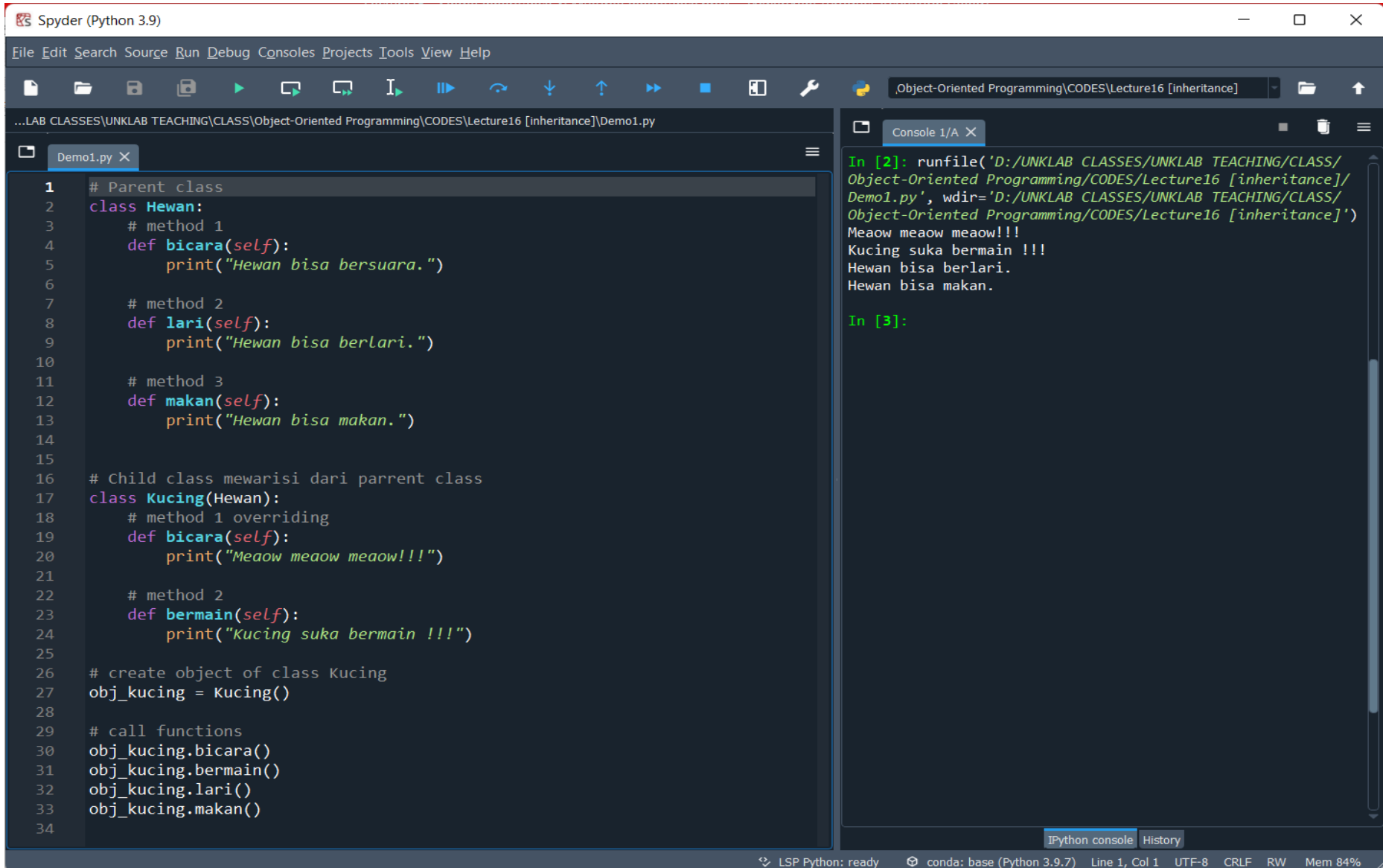
Single Inheritance

In inheritance, the child class can access all the methods defined in the parent class. In addition, the child class can include its own specific methods.

Konsep **single inheritance** hanya memperbolehkan suatu *child class* dan mempunyai satu *parent class*.

Single Inheritance yaitu satu *parent class* yang mewariskan **variables** dan **methods**-nya hanya kepada satu **child class**.

Single Inheritance



The image shows the Spyder Python IDE interface. The main editor window displays a Python script named `Demo1.py` illustrating single inheritance. The script defines a parent class `Hewan` with three methods: `bicara`, `lari`, and `makan`. A child class `Kucing` inherits from `Hewan` and overrides the `bicara` method. The script also creates an instance of `Kucing` and calls its methods.

```
1 # Parent class
2 class Hewan:
3     # method 1
4     def bicara(self):
5         print("Hewan bisa bersuara.")
6
7     # method 2
8     def lari(self):
9         print("Hewan bisa berlari.")
10
11    # method 3
12    def makan(self):
13        print("Hewan bisa makan.")
14
15 # Child class mewarisi dari parent class
16 class Kucing(Hewan):
17     # method 1 overriding
18     def bicara(self):
19         print("Meaow meaow meaow!!!")
20
21    # method 2
22    def bermain(self):
23        print("Kucing suka bermain !!!")
24
25 # create object of class Kucing
26 obj_kucing = Kucing()
27
28 # call functions
29 obj_kucing.bicara()
30 obj_kucing.bermain()
31 obj_kucing.lari()
32 obj_kucing.makan()
33
34
```

The IPython console on the right shows the output of running the script:

```
In [2]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture16 [inheritance]/
Demo1.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture16 [inheritance]')
Meaow meaow meaow!!!
Kucing suka bermain !!!
Hewan bisa berlari.
Hewan bisa makan.

In [3]:
```

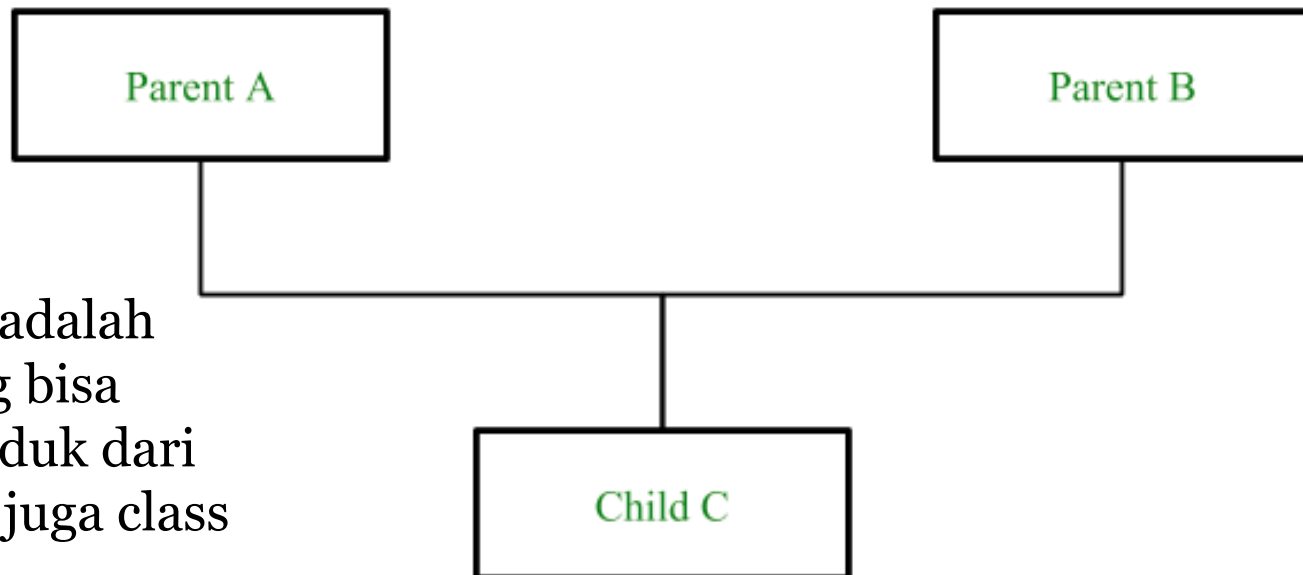
The status bar at the bottom indicates the environment is LSP Python: ready, conda: base (Python 3.9.7), and the current file is Line 1, Col 1, UTF-8, CRLF, RW, with 84% memory usage.

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green with some showing signs of aging or damage. A large, faint watermark is visible across the center of the slide.

2 - Multiple Inheritance

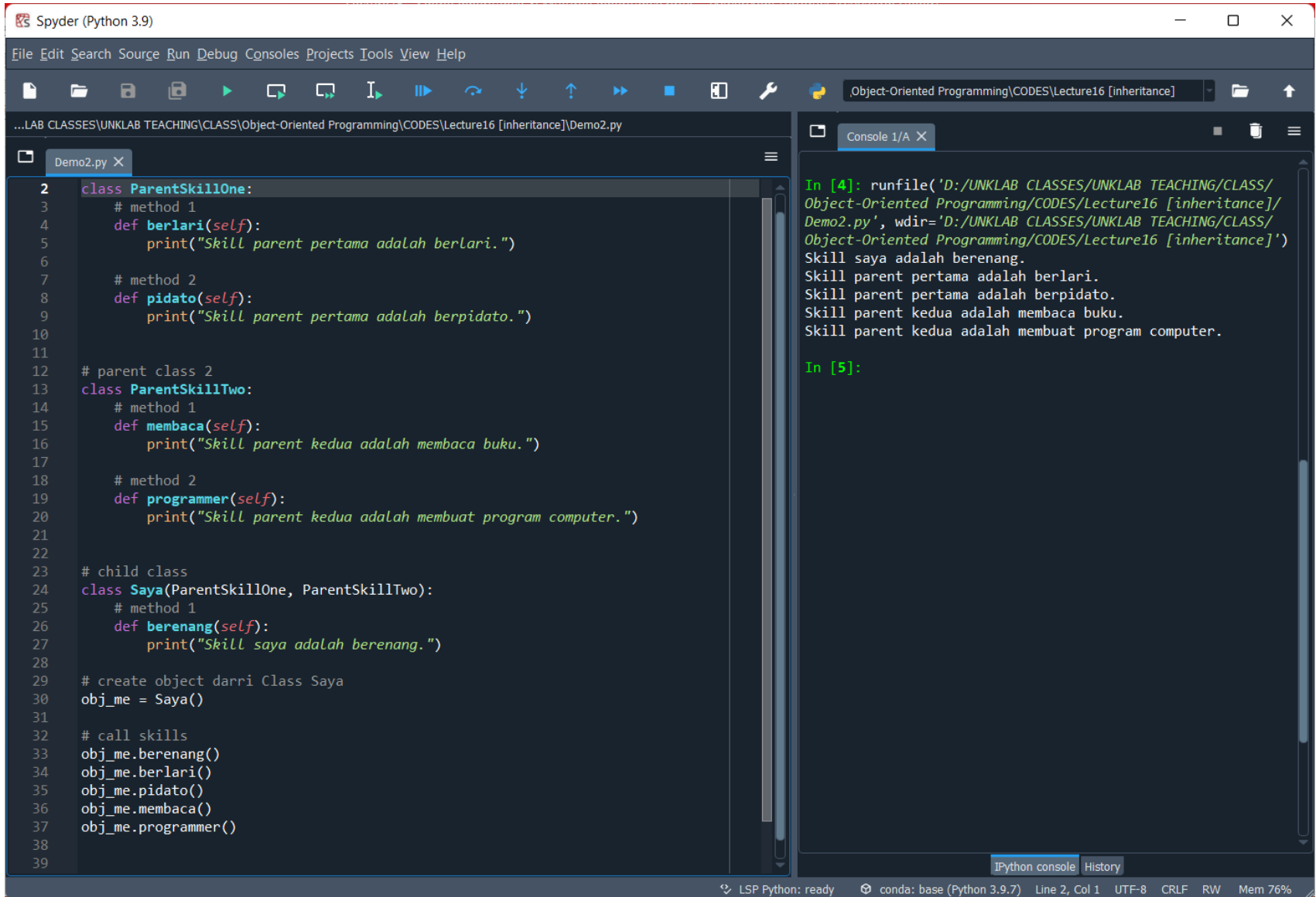
Multiple Inheritance

- ❖ Multiple inheritance allows one child class to inherit from more than one parent class.
- ❖ Multiple Inheritance is a class inheritance where a class can have more than one parent class directly.
- ❖ Multiple Inheritance is a class inheritance where a class can have more than one parent class directly.



Contohnya adalah class C yang bisa memiliki induk dari class A dan juga class B sekaligus.

Multiple Inheritance



The image shows the Spyder Python IDE interface. The left pane displays a Python script named `Demo2.py` with the following code:

```
2 class ParentSkillOne:
3     # method 1
4     def berlari(self):
5         print("Skill parent pertama adalah berlari.")
6
7     # method 2
8     def pidato(self):
9         print("Skill parent pertama adalah berpidato.")
10
11 # parent class 2
12 class ParentSkillTwo:
13     # method 1
14     def membaca(self):
15         print("Skill parent kedua adalah membaca buku.")
16
17     # method 2
18     def programmer(self):
19         print("Skill parent kedua adalah membuat program computer.")
20
21 # child class
22 class Saya(ParentSkillOne, ParentSkillTwo):
23     # method 1
24     def berenang(self):
25         print("Skill saya adalah berenang.")
26
27 # create object dari Class Saya
28 obj_me = Saya()
29
30 # call skills
31 obj_me.berenang()
32 obj_me.berlari()
33 obj_me.pidato()
34 obj_me.membaca()
35 obj_me.programmer()
36
37
38
39
```

The right pane shows the IPython console output for two input prompts:

```
In [4]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture16 [inheritance]/
Demo2.py', wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture16 [inheritance]')
Skill saya adalah berenang.
Skill parent pertama adalah berlari.
Skill parent pertama adalah berpidato.
Skill parent kedua adalah membaca buku.
Skill parent kedua adalah membuat program computer.

In [5]:
```

The status bar at the bottom indicates: LSP Python: ready, conda: base (Python 3.9.7), Line 2, Col 1, UTF-8, CRLF, RW, Mem 76%.

Perhatikan code di samping, class ParentSkillOne dan ParentSkillTwo memiliki method yang sama.

Apa yang terjadi jika terdapat method yang sama di kelas-kelas parent yang diwarisi dalam multiple inheritance? Error konflik? Override?

Skill pidato dari parent class mana yang akan diwariskan?

```
# parent class 1
class ParentSkillOne:
    # method 1
    def berlari(self):
        print("Skill parent pertama adalah berlari.")

    # method 2
    def pidato(self):
        print("Skill parent pertama adalah berpidato.")

# parent class 2
class ParentSkillTwo:
    # method 1
    def membaca(self):
        print("Skill parent kedua adalah membaca buku.")

    # method 2
    def pidato(self):
        print("Skill parent kedua adalah berpidato.")

# child class
class Saya(ParentSkillOne, ParentSkillTwo):
    # method 1
    def programmer(self):
        print("Skill saya adalah berenang.")

# create object dari Class Saya
obj_me = Saya()

# call skills
obj_me.berlari()
obj_me.pidato()
obj_me.membaca()
obj_me.programmer()
```

```
1 # parent class 1
2 class ParentSkillOne:
3     # method 1
4     def berlari(self):
5         print("Skill parent pertama adalah berlari.")
6
7     # method 2
8     def pidato(self):
9         print("Skill parent pertama adalah berpidato.")
10
11
12 # parent class 2
13 class ParentSkillTwo:
14     # method 1
15     def membaca(self):
16         print("Skill parent kedua adalah membaca buku.")
17
18     # method 2
19     def pidato(self):
20         print("Skill parent kedua adalah berpidato.")
21
22
23 # child class
24 class Saya(ParentSkillOne, ParentSkillTwo):
25     # method 1
26     def programmer(self):
27         print("Skill saya adalah berenang.")
28
29 # create object dari Class Saya
30 obj_me = Saya()
31
32 # call skills
33 obj_me.berlari()
34 obj_me.pidato()
35 obj_me.membaca()
36 obj_me.programmer()
37
```

```
In [13]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented
Programming/CODES/Lecture15 [inheritance]/Demo3.py',
wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/
CODES/Lecture15 [inheritance]')
Skill parent pertama adalah berlari.
Skill parent pertama adalah berpidato.
Skill parent kedua adalah membaca buku.
Skill saya adalah berenang.
```

```
In [14]:
```

```

1 # parent class 1
2 class ParentSkillOne:
3     # method 1
4     def berlari(self):
5         print("Skill parent pertama adalah berlari.")
6
7     # method 2
8     def pidato(self):
9         print("Skill parent pertama adalah berpidato.")
10
11
12 # parent class 2
13 class ParentSkillTwo:
14     # method 1
15     def membaca(self):
16         print("Skill parent kedua adalah membaca buku.")
17
18     # method 2
19     def pidato(self):
20         print("Skill parent kedua adalah berpidato.")
21
22
23 # child class
24 class Saya(ParentSkillOne, ParentSkillTwo):
25     # method 1
26     def pidato(self):
27         print("Skill saya adalah berpidato.")
28
29 # create object dari Class Saya
30 obj_me = Saya()
31
32 # call skills
33 obj_me.berlari()
34 obj_me.pidato()
35 obj_me.membaca()

```

Apa yang terjadi jika
terdapat method yang sama
di kelas-kelas parent dan
bahkan di class child?
Error? konflik? Override?

Skill pidato class mana yang
akan digunakan?

```
1 # parent class 1
2 class ParentSkillOne:
3     # method 1
4     def berlari(self):
5         print("Skill parent pertama adalah berlari.")
6
7     # method 2
8     def pidato(self):
9         print("Skill parent pertama adalah berpidato.")
10
11
12 # parent class 2
13 class ParentSkillTwo:
14     # method 1
15     def membaca(self):
16         print("Skill parent kedua adalah membaca buku.")
17
18     # method 2
19     def pidato(self):
20         print("Skill parent kedua adalah berpidato.")
21
22
23 # child class
24 class Saya(ParentSkillOne, ParentSkillTwo):
25     # method 1
26     def pidato(self):
27         print("Skill saya adalah berpidato.")
28
29 # create object dari Class Saya
30 obj_me = Saya()
31
32 # call skills
33 obj_me.berlari()
34 obj_me.pidato()
35 obj_me.membaca()
36
37
```

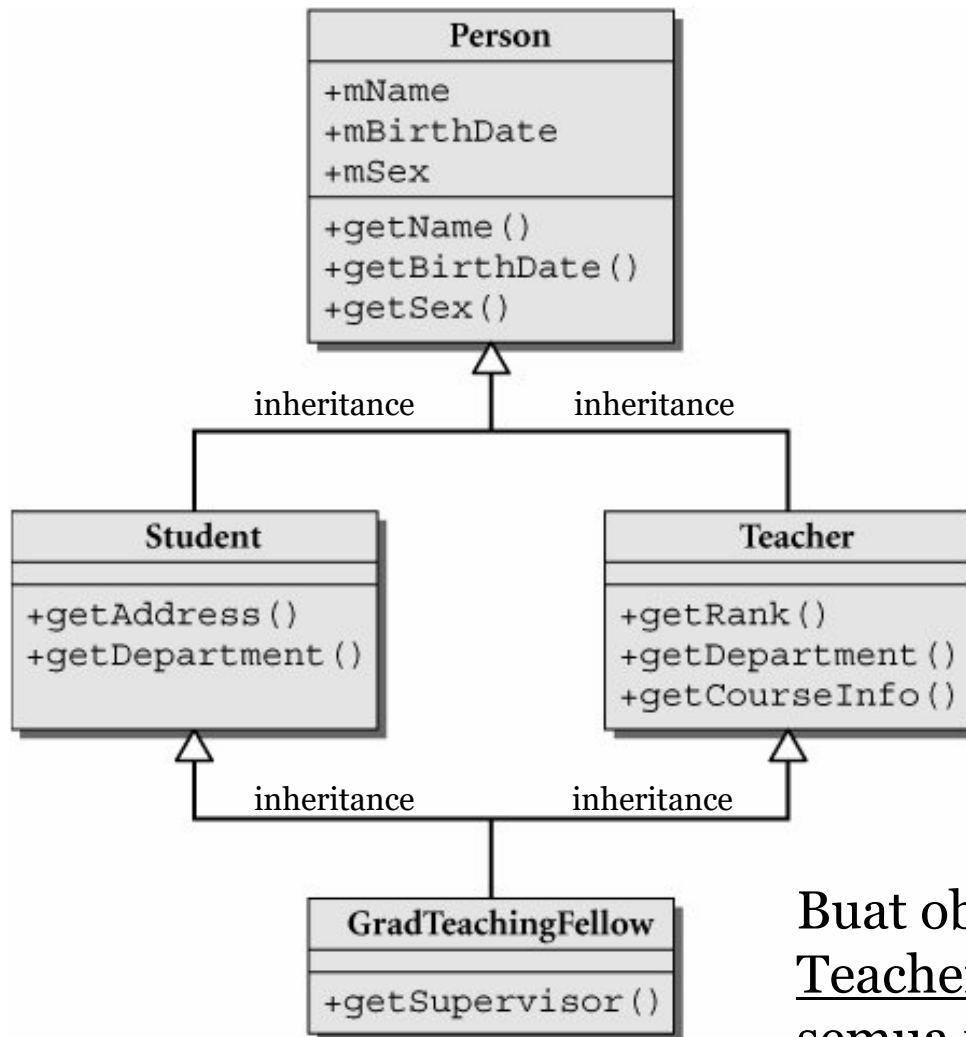
```
In [14]: runfile('D:/UNKLAB CLASSES/CLASS/Object-Oriented
Programming/COD\ES/Lecture15 [inheritance]/Demo3.py',
wdir='D:/UNKLAB CLASSES/CLASS/Object-Oriented Programming/
COD\ES/Lecture15 [inheritance]')
Skill parent pertama adalah berlari.
Skill saya adalah berpidato.
Skill parent kedua adalah membaca buku.

In [15]: |
```



Exercise *for* Students

Convert class diagram below into Python code. In class diagram, the + (plus) symbol is used to indicate attributes or methods that are public.



Ada berapa banyak
method yang bisa
diakses oleh object
dari class
GradTeachingFellow?

Buat object dari class Person, Student, Teacher, dan GradTeachingFellow dan akses semua method yang mungkin bisa?

- 1) Apa yang dimaksud dengan “multiple inheritance” dalam OOP?
- 2) Apa keuntungan menggunakan *multiple inheritance* dalam pengembangan software?
- 3) Bagaimana cara mendefinisikan kelas yang menggunakan multiple inheritance dalam Python?
- 4) Apa perbedaan antara *single inheritance* dan *multiple inheritance*?
- 5) Apa yang terjadi jika terdapat *method* yang sama di kelas-kelas *parent* yang diwarisi dalam *multiple inheritance*?

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

